

SIT313 P2 Welcome Email Report

Student: Romil Bijarnia

Task: P2 Welcome Email

Application: DEV@Deakin React and Express project

Submission files: report.html, report.pdf, server.js, index.html, .env.example, supporting-frontend-src/

Overview

For this task I added a newsletter sign-up feature to the DEV@Deakin platform and built the Node.js backend that turns each new sign-up into a welcome email. The frontend renders a “Sign up for our daily insider” panel on the home page, and an ExpressJS API receives the subscription, validates the email address, stores the subscriber on the mailing list, and sends the welcome email asynchronously through an email API before confirming success back to the browser. I verified the whole flow by starting the submitted server and exercising every endpoint path from a terminal: a new subscription returns Status 200 with the welcome-email metadata, a repeated email returns 409, and malformed input returns 400. The transcripts of those runs are included below, together with a screenshot of the running platform.

1. GitLab repository

The welcome-email backend lives in the same Git repository as the rest of the cumulative DEV@Deakin project, so every file in this submission is tracked alongside the frontend it serves. The repository follows the hygiene rules from the task sheet: a `.gitignore` at the root excludes `node_modules/`, the Vite `dist/` build output, and, importantly for this task, `.env` and `.env.local`, so the SendGrid API key can never be committed. The generated mailing list (`server/data/mailling-list.json`) is ignored as well, because subscriber email addresses are personal data and do not belong in version-control history. In place of a real `.env`, the repository carries a committed `.env.example` (included in this submission) that documents the four supported settings without any secret values:

```
PORT=5050
FRONTEND_ORIGIN=http://127.0.0.1:5173
SENDGRID_API_KEY=
SENDGRID_FROM=
```

`server.js` loads these at start-up with `require('dotenv').config()`, reads the port and allowed frontend origin from `process.env`, and only ever touches the SendGrid credentials through `process.env.SENDGRID_API_KEY` and `process.env.SENDGRID_FROM`. No key, address, or other sensitive value appears anywhere in the code.

2. Sign-Up Feature

The sign-up feature is the `Newsletter` component in `supporting-frontend-src/App.jsx`, rendered on the home page between the tutorial cards and the photo gallery so it matches the “SIGN UP FOR OUR DAILY INSIDER” layout from the task sheet. It is a plain HTML form styled with hand-written CSS (`styles.css`): the panel is a two-column CSS grid with the heading on the left and the form on the right, and the form itself is a flexbox row that keeps the email field and Subscribe button together — the two layout models suggested in the task sheet’s Optional Technologies section. Because the platform had already grown into a React application by this task, the form is managed as a controlled component: the input value lives in `useState`, the field is declared as `type="email"` with `required` so the browser applies its first layer of validation, and an `aria-label` keeps it accessible.

Submitting the form calls `subscribeToNewsletter(email)` from `supporting-frontend-src/api.js`, which sends the address to the backend with the `fetch()` API as JSON — the second of the two request options the task sheet offers. While the request is in flight the button is disabled and its label switches to “Sending”, and when the response arrives the component prints the outcome under the form, for example *Status 200: Subscription successful. Welcome email sent*. The helper never throws: if the Express service is not running, it resolves to a friendly error object so the form resets its loading state and tells the user to start the backend instead of hanging. The base URL comes from the `VITE_API_BASE` environment variable with a `http://127.0.0.1:5050` development default, keeping deployment configuration out of the source. React renders the returned message through normal text interpolation, so the server response is automatically escaped and cannot inject markup into the page.

3. ExpressJS Backend

The backend is `server.js`, a Node.js service built with Express 4 (`express ^4.21.2`) plus `cors ^2.8.5` and `dotenv ^16.4.7`. It starts with `node server.js` (in the full project, `npm run server`, or `npm run dev` to launch the Vite frontend at the same time through `concurrently`; during development I also used nodemon-style restarts by re-running the script after each change). Two middleware layers are registered before the routes: `cors()` restricted to an allow-list containing the configured frontend origin, the Vite dev server (`http://localhost:5173`), and the Vite preview port (`http://127.0.0.1:4173`), and `express.json()` with a 1 MB body limit so oversized payloads are rejected before any handler runs.

Handling sign-ups

POST /api/subscribe receives { email } from the form. The handler normalises the address (trim plus lowercase), then validates it with a helper that enforces a maximum length of 254 characters (the practical RFC 5321 limit) and a name@domain.tld pattern; failures return 400 with a clear message. The mailing list is loaded from data/mailling-list.json — created on demand through fs/promises with mkdir(..., { recursive: true }) — and if the address is already present the handler answers 409 Conflict so the frontend can tell the user they are already subscribed. Only when the address is new does the server send the welcome email, append the subscriber record (id, email, timestamp, email provider, status, and message id) to the JSON file, log the result to the terminal, and answer 200.

Sending the welcome email asynchronously

Email delivery is isolated in the sendWelcomeEmail() function, and the route handler awaits it inside an async function, so the Node event loop stays free to serve other requests while the email API round-trip is in flight. When SENDGRID_API_KEY and SENDGRID_FROM are present, the function calls SendGrid's v3 Mail Send endpoint (https://api.sendgrid.com/v3/mail/send) with the native fetch() client, passing the key as a Bearer token in the Authorization header and a JSON body containing the personalisation, the verified sender, the subject "Welcome to DEV@Deakin", and a plain-text welcome message. A non-2xx reply is converted into a thrown error carrying SendGrid's status so the route can fail loudly, and a successful reply returns the provider name, the HTTP status, and SendGrid's x-message-id header. When no key is configured — the safe default for a marked submission — the function falls back to a local provider that simulates the API latency and returns status 200, which keeps the full flow demonstrable without ever distributing a live credential. The email is sent *before* the subscriber is written to the mailing list, so a failed delivery cannot leave a phantom subscriber behind.

HTTP response status codes

Route	Outcome	Status
POST /api/subscribe	New subscriber saved, welcome email sent	200
POST /api/subscribe	Missing, malformed, or over-length email	400
POST /api/subscribe	Email already on the mailing list	409
POST /api/subscribe	Unexpected failure (details logged server-side only)	500
GET /api/health	Service liveness check	200
GET /api/subscribers	Mailing-list evidence during development	200

POST /api/subscriptions	Paid-plan subscription created (later D2 feature, same backend)	201 / 400 / 409
----------------------------	---	-----------------

Every error response is a small JSON object with a human-readable message and the status code. Internal failures deliberately return a generic message while the real error is written to the server log with `console.error`, so stack traces, file paths, and email-API responses are never leaked to the browser. The `GET /api/subscribers` endpoint exists as development evidence that sign-ups really land on the mailing list; in a production deployment it would sit behind authentication because it exposes subscriber addresses.

Screenshot of the DEV@Deakin Platform

SIT313 SECURE FRONTEND APPLICATIONS

DEV@Deakin, built as one complete full- stack learning platform.

A cumulative React application with portfolio content, article and tutorial cards, authentication, posting, browsing, newsletter integration, and advanced HD features.

Create a post

Browse posts



Here is what I have done so far

I have built the DEV@Deakin platform from a static portfolio into a connected app with route-based pages, reusable React components, API communication, local data persistence, form validation, and secure-authentication concepts.

FEATURED ARTICLES

Articles rendered from an array with map



Designing Secure React Interfaces

Practical patterns for forms, validation, routing, and safe user feedback.

☆ 5.0 Romil Bijarnia

Connecting APIs to Frontend Pages

How frontend forms send requests, handle responses, and report errors clearly.

☆ 4.9 DEV@Deakin

Building a Newsletter Signup

A small full-stack feature that turns a static site into a connected app.

☆ 4.8 Romil Bijarnia

FEATURED TUTORIALS

Tutorials with consistent media cards



React Routing in DEV@Deakin

Create pages for Home, Post, Login, Sign-Up, Browse Posts, and HD features.

☆ 5.0 Romil Bijarnia



Saving and Browsing Posts

Collect form data, validate it, store it, filter it, and expand previews.

☆ 4.9 DEV@Deakin



Authentication UI Basics

Use reusable components, user records, password hashing, and redirect flows.

☆ 4.7 Romil Bijarnia

Sign up for our daily insider

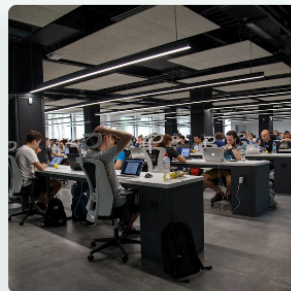
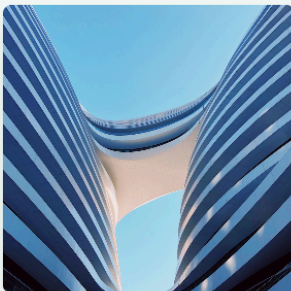
Subscribe to receive DEV@Deakin updates through the Express newsletter backend.

[Subscribe](#)

Status 200: Subscription successful. Welcome email sent.

RESPONSIVE GALLERY

My Photos



DEV@Deakin

Created by Romil Bijarnia for SIT313 Secure Frontend Applications.

[Home](#) [Post](#) [Browse Posts](#) [HD Features](#)

The running DEV@Deakin home page. The “Sign up for our daily insider” panel sits below the tutorial cards; the message under the form is the live backend response after subscribing: “Status 200: Subscription successful. Welcome email sent.”

Terminal Evidence of the Email API Status 200

The first transcript is the welcome-email smoke test against the submitted `server.js`. The opening line is the server's own `console.log()` confirming the email API returned Status 200 for the queued welcome email, followed by the full JSON response the frontend receives:

```
Status 200: welcome email queued for romil.demo.1783562025753@example.com using Local
SIT313 email provider
Newsletter status 200: Subscription successful. Welcome email sent.
{
  "message": "Subscription successful. Welcome email sent.",
  "status": 200,
  "subscriber": {
    "id": "sub-1783562025970",
    "email": "romil.demo.1783562025753@example.com",
    "createdAt": "2026-07-09T01:53:45.970Z",
    "emailProvider": "Local SIT313 email provider",
    "emailStatus": 200,
    "emailMessageId": "local-1783562025970"
  }
}
```

The second transcript exercises every response path of the backend from the terminal — health check, fresh subscription, duplicate, invalid input, over-length input, missing field, mailing-list read, and the shared subscription-plan endpoint — confirming each returns the status code documented above:

```
PASS GET /api/health: expected 200, got 200
Status 200: welcome email queued for romil.p2.1783561948506@example.com using Local
SIT313 email provider
PASS POST /api/subscribe new email: expected 200, got 200
  message: Subscription successful. Welcome email sent.
  provider: Local SIT313 email provider, emailStatus: 200
PASS POST /api/subscribe duplicate email: expected 409, got 409
  message: This email is already subscribed.
PASS POST /api/subscribe invalid email: expected 400, got 400
  message: Enter a valid email address.
PASS POST /api/subscribe 262-char email: expected 400, got 400
PASS POST /api/subscribe missing email: expected 400, got 400
PASS GET /api/subscribers: expected 200, got 200
  mailing list count: 1
PASS POST /api/subscriptions valid plan: expected 201, got 201
```

```
PASS POST /api/subscriptions 200-char name: expected 400, got 400
ALL CHECKS PASSED
```

Relevant Source Files

- `server.js` — the ExpressJS backend (in the full repository: `server/server.js`).
- `.env.example` — documented environment settings; the real `.env` is git-ignored.
- `supporting-frontend-src/App.jsx` — the Newsletter React component with the sign-up form.
- `supporting-frontend-src/api.js` — the `fetch()` client for `POST /api/subscribe`.
- `supporting-frontend-src/styles.css` — the hand-written CSS, including the newsletter panel layout.
- `index.html` — the Vite entry page that mounts the React application.

Conclusion

The task gave me a complete, verified sign-up-to-welcome-email pipeline: an accessible form on the DEV@Deakin home page, an Express API that validates input on top of the browser's checks, duplicate protection with meaningful status codes, asynchronous email delivery behind an environment-based credential boundary, and persisted mailing-list records that stay out of Git. Working through it made the request-response contract between a React frontend and a Node.js backend concrete for me — especially how much of a secure frontend actually depends on the server telling the truth with its status codes while keeping its internals to itself.